

```

1 //#####
2 // FILE:   FinalProjectStarter_main.c
3 //
4 // TITLE:  Final Project Starter
5 //#####
6
7 // Included Files
8 #include <stdio.h>
9 #include <stdlib.h>
10 #include <stdarg.h>
11 #include <string.h>
12 #include <math.h>
13 #include <limits.h>
14 #include "F28x_Project.h"
15 #include "F2837xD_SWPrioritizedIsrLevels.h"
16 #include "driverlib.h"
17 #include "device.h"
18 #include "F28379dSerial.h"
19 #include "song.h"
20 #include "dsp.h"
21 #include "fpu32/fpu_rfft.h"
22 #include "xy.h"
23 #include "MatrixMath.h"
24 #include "SE423Lib.h"
25 #include "OptiTrack.h"
26
27 #define PI          3.1415926535897932384626433832795
28 #define TWOPI       6.283185307179586476925286766559
29 #define HALFPi      1.5707963267948966192313216916398
30 // The Launchpad's CPU Frequency set to 200 you should not change this value
31 #define LAUNCHPAD_CPU_FREQUENCY 200
32
33 // ----- code for CAN start here -----
34 #include "F28379dCAN.h"
35 // #define TX_MSG_DATA_LENGTH 4
36 // #define TX_MSG_OBJ_ID 0 //transmit
37
38 #define RX_MSG_DATA_LENGTH 8
39 #define RX_MSG_OBJ_ID_1 1 //measurement from sensor 1
40 #define RX_MSG_OBJ_ID_2 2 //measurement from sensor 2
41 #define RX_MSG_OBJ_ID_3 3 //quality from sensor 1
42 #define RX_MSG_OBJ_ID_4 4 //quality from sensor 2
43 // ----- code for CAN end here -----
44
45 // Interrupt Service Routines predefinition
46 __interrupt void cpu_timer0_isr(void);
47 __interrupt void cpu_timer1_isr(void);
48 __interrupt void cpu_timer2_isr(void);
49 __interrupt void SWI1_HighestPriority(void);
50 __interrupt void SWI2_MiddlePriority(void);
51 __interrupt void SWI3_LowestPriority(void);
52 __interrupt void ADC12_ISR(void);
53 __interrupt void SPIB_isr(void);
54 // ----- code for CAN start here -----
55 __interrupt void can_isr(void);
56 // ----- code for CAN end here -----
57
58 void setF28027EPWM1A(float controleffort);
59 int16_t EPwm1A_F28027 = 1500;
60 void setF28027EPWM2A(float controleffort);
61 int16_t EPwm2A_F28027 = 1500;
62
63 // ----- code for CAN start here -----
64 // volatile uint32_t txMsgCount = 0;
65 // extern uint16_t txMsgData[4];
66
67 volatile uint32_t rxMsgCount_1 = 0;
68 volatile uint32_t rxMsgCount_2 = 0;
69 extern uint16_t rxMsgData[8];
70
71 uint32_t dis_raw_1[2];
72 uint32_t dis_raw_2[2];
73 uint32_t dis_1 = 0;
74 uint32_t dis_2 = 0;
75
76 uint32_t quality_raw_1[4];
77 uint32_t quality_raw_2[4];
78 float quality_1 = 0.0;
79 float quality_2 = 0.0;
80
81 uint32_t lightlevel_raw_1[4];
82 uint32_t lightlevel_raw_2[4];
83 float lightlevel_1 = 0.0;
84 float lightlevel_2 = 0.0;
85
86 uint32_t measure_status_1 = 0;
87 uint32_t measure_status_2 = 0;
88
89 volatile uint32_t errorFlag = 0;
90 // ----- code for CAN end here -----
91
92 uint32_t numTimer0calls = 0;
93 uint16_t UARTPrint = 0;
94
95 float printLV1 = 0;
96 float printLV2 = 0;
97
98 float printLinux1 = 0;
99 float printLinux2 = 0;
100
101 uint16_t LADARi = 0;
102 char G_command[] = "G04472503\n"; //command for getting distance -120 to 120 degree
103 uint16_t G_len = 11; //length of command
104 xy_ladar_pts[228]; //xy data
105 float LADARrightfront = 0;
106 float LADARfront = 0;
107 float LADARtemp_x = 0;
108 float LADARtemp_y = 0;
109 extern datapts_ladar_data[228];
110
111 extern uint16_t newLinuxCommands;
112 extern float LinuxCommands[CMDNUM_FROM_FLOATS];

```

```

113
114 extern uint16_t NewLVData;
115 extern float fromLVvalues[LVNUM_TOFROM_FLOATS];
116 extern LVSendFloats_t DataToLabView;
117 extern char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
118 extern uint16_t LADARpingpong;
119 extern uint16_t NewCAMDataThreshold1; // Flag new data
120 extern float fromCAMvaluesThreshold1[CAMNUM_FROM_FLOATS];
121 extern uint16_t NewCAMDataThreshold2; // Flag new data
122 extern float fromCAMvaluesThreshold2[CAMNUM_FROM_FLOATS];
123
124 extern uint16_t NewCAMDataAprilTag1; // Flag new data
125 extern float fromCAMvaluesAprilTag1[CAMNUM_FROM_FLOATS];
126
127 float tagid = 0;
128 float tagx = 0;
129 float tagy = 0;
130 float tagz = 0;
131 float tagthetax = 0;
132 float tagthetay = 0;
133 float tagthetaz = 0;
134 int32_t numtag = 0;
135 int32_t AprilTagState = 10;
136 float KpAprilTag = -1.0;
137 float AprilTagSetPoint = 0;
138 int32_t AprilTagCount = 0;
139
140 float MaxAreaThreshold1 = 0;
141 float MaxColThreshold1 = 0;
142 float MaxRowThreshold1 = 0;
143 float NextLargestAreaThreshold1 = 0;
144 float NextLargestColThreshold1 = 0;
145 float NextLargestRowThreshold1 = 0;
146 float NextNextLargestAreaThreshold1 = 0;
147 float NextNextLargestColThreshold1 = 0;
148 float NextNextLargestRowThreshold1 = 0;
149
150 float MaxAreaThreshold2 = 0;
151 float MaxColThreshold2 = 0;
152 float MaxRowThreshold2 = 0;
153 float NextLargestAreaThreshold2 = 0;
154 float NextLargestColThreshold2 = 0;
155 float NextLargestRowThreshold2 = 0;
156 float NextNextLargestAreaThreshold2 = 0;
157 float NextNextLargestColThreshold2 = 0;
158 float NextNextLargestRowThreshold2 = 0;
159
160 // cja Global variables to hold camera depth for orange and green objects
161 float odist = 0.0;
162 float gdist = 0.0;
163
164 // cja Global error variable for P controller to track colored objects
165 float colcentroid = 0.0;
166
167 // cja P colored object tracking controller gain
168 float kpvision = 0.0;
169
170 // cja Global timer variables
171 uint32_t count22 = 0; // cja Timing between states to pick up golf balls
172 uint32_t count1 = 0; // cja Delay when resetting back to state 1
173
174 uint32_t numThres1 = 0;
175 uint32_t numThres2 = 0;
176
177 pose ROBOTps = {0,0,0}; //robot position
178 pose LADARps = {3.5/12,0,0,1}; // 3.5/12 for front mounting, theta is not used in this current code
179 float LADARxoffset = 0;
180 float LADARyoffset = 0;
181
182 uint32_t timecount = 0;
183 int16_t RobotState = 1;
184 int16_t checkfronttally = 0;
185 int32_t WallFollowtime = 0;
186
187 #define NUMWAYPOINTS 10
188 uint16_t statePos = 0;
189 pose robotdest[NUMWAYPOINTS]; // array of waypoints for the robot
190 uint16_t i = 0; //for loop
191
192 uint16_t right_wall_follow_state = 2; // right follow
193 float Kp_front_wall = -2.0;
194 float front_turn_velocity = 0.2;
195 float left_turn_Stop_threshold = 3.5;
196 float Kp_right_wal = -4.0;
197 float ref_right_wall = 1.1;
198 float foward_velocity = 1.0;
199 float left_turn_Start_threshold = 1.3;
200 float turn_saturation = 2.5;
201
202 float x_pred[3][1] = {{0},{0},{0}}; // predicted state
203
204 //more kalman vars
205 float B[3][2] = {{1,0},{1,0},{0,1}}; // control input model
206 float u[2][1] = {{0},{0}}; // control input in terms of velocity and angular velocity
207 float Bu[3][1] = {{0},{0},{0}}; // matrix multiplication of B and u
208 float z[3][1]; // state measurement
209 float eye3[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // 3x3 identity matrix
210 float K[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // optimal Kalman gain
211 #define ProcUncert 0.0001
212 #define CovScalar 10
213 float Q[3][3] = {{ProcUncert,0,ProcUncert/CovScalar},
214                 {0,ProcUncert,ProcUncert/CovScalar},
215                 {ProcUncert/CovScalar,ProcUncert/CovScalar,ProcUncert}}; // process noise (covariance of encoders and gyro)
216 #define MeasUncert 1
217 float R[3][3] = {{MeasUncert,0,MeasUncert/CovScalar},
218                 {0,MeasUncert,MeasUncert/CovScalar},
219                 {MeasUncert/CovScalar,MeasUncert/CovScalar,MeasUncert}}; // measurement noise (covariance of OptiTrack Motion Capture measurement)
220 float S[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance
221 float S_inv[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // innovation covariance matrix inverse
222 float P_pred[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // predicted covariance (measure of uncertainty for current position)
223 float temp_3x3[3][3]; // intermediate storage
224 float temp_3x1[3][1]; // intermediate storage
225 float ytilde[3][1]; // difference between predictions
226

```

```

227 // Optitrack Variables
228 int32_t OptiNumUpdatesProcessed = 0;
229 pose OPTITRACKps;
230 extern uint16_t new_optitrack;
231 extern float Optitrackdata[OPTITRACKDATASIZE];
232 int16_t newOPTITRACKpose=0;
233
234 int16_t adcc2result = 0;
235 int16_t adcc3result = 0;
236 int16_t adcc4result = 0;
237 int16_t adcc5result = 0;
238 float adcc2Volt = 0.0;
239 float adcc3Volt = 0.0;
240 float adcc4Volt = 0.0;
241 float adcc5Volt = 0.0;
242 int32_t numADCCcalls = 0;
243 float LeftWheel = 0;
244 float RightWheel = 0;
245 float LeftWheel_1 = 0;
246 float RightWheel_1 = 0;
247 float LeftVel = 0;
248 float RightVel = 0;
249 float uLeft = 0;
250 float uRight = 0;
251 float HandValue = 0;
252 float vref = 0;
253 float turn = 0;
254 float gyro9250_drift = 0;
255 float gyro9250_angle = 0;
256 float old_gyro9250 = 0;
257 float gyroLPR510_angle = 0;
258 float gyroLPR510_offset = 0;
259 float gyroLPR510_drift = 0;
260 float old_gyroLPR510 = 0;
261 float gyro9250_radians = 0;
262 float gyroLPR510_radians = 0;
263
264 int16_t readdata[25];
265 int16_t IMU_data[9];
266
267 float accelx = 0;
268 float accely = 0;
269 float accelz = 0;
270 float gyrox = 0;
271 float gyroy = 0;
272 float gyroz = 0;
273
274 // Needed global Variables
275 float accelx_offset = 0;
276 float accely_offset = 0;
277 float accelz_offset = 0;
278 float gyrox_offset = 0;
279 float gyroy_offset = 0;
280 float gyroz_offset = 0;
281 int16_t calibration_state = 0;
282 int32_t calibration_count = 0;
283 int16_t doneCal = 0;
284
285 #define MPU9250 1
286 #define DAN28027 2
287 int16_t CurrentChip = MPU9250;
288 int16_t DAN28027Garbage = 0;
289 int16_t dan28027adc1 = 0;
290 int16_t dan28027adc2 = 0;
291 uint16_t MPU9250ignoreCNT = 0; //This is ignoring the first few interrupts if ADCC_ISR and start sending to IMU after these first few interrupts.
292
293 float RCangle;
294
295 void main(void)
296 {
297     // PLL, WatchDog, enable Peripheral Clocks
298     // This example function is found in the F2837xD_SysCtrl.c file.
299     InitSysCtrl();
300
301     InitGpio();
302
303     InitSE423DefaultGPIO();
304
305     //Scope for Timing
306     GPIO_SetupPinMux(11, GPIO_MUX_CPU1, 0);
307     GPIO_SetupPinOptions(11, GPIO_OUTPUT, GPIO_PUSHPULL);
308     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
309
310     GPIO_SetupPinMux(32, GPIO_MUX_CPU1, 0);
311     GPIO_SetupPinOptions(32, GPIO_OUTPUT, GPIO_PUSHPULL);
312     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
313
314     GPIO_SetupPinMux(52, GPIO_MUX_CPU1, 0);
315     GPIO_SetupPinOptions(52, GPIO_OUTPUT, GPIO_PUSHPULL);
316     GpioDataRegs.GPBCLEAR.bit.GPIO52 = 1;
317
318     GPIO_SetupPinMux(60, GPIO_MUX_CPU1, 0);
319     GPIO_SetupPinOptions(60, GPIO_OUTPUT, GPIO_PUSHPULL);
320     GpioDataRegs.GPBCLEAR.bit.GPIO60 = 1;
321
322     GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
323     GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSHPULL);
324     GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
325
326     GPIO_SetupPinMux(67, GPIO_MUX_CPU1, 0);
327     GPIO_SetupPinOptions(67, GPIO_OUTPUT, GPIO_PUSHPULL);
328     GpioDataRegs.GPCCLEAR.bit.GPIO67 = 1;
329
330 // ----- code for CAN start here -----
331 //GPIO17 - CANRXB
332 GPIO_SetupPinMux(17, GPIO_MUX_CPU1, 2);
333 GPIO_SetupPinOptions(17, GPIO_INPUT, GPIO_ASYNC);
334
335 //GPIO12 - CANTXB
336 GPIO_SetupPinMux(12, GPIO_MUX_CPU1, 2);
337 GPIO_SetupPinOptions(12, GPIO_OUTPUT, GPIO_PUSHPULL);
338 // ----- code for CAN end here -----
339
340

```

```

341
342 // ----- code for CAN start here -----
343 // Initialize the CAN controller
344 InitCANB();
345
346 // Set up the CAN bus bit rate to 1000 kbps
347 setCANBitRate(200000000, 1000000);
348
349 // Enables Interrupt line 0, Error & Status Change interrupts in CAN_CTL register.
350 CanbRegs.CAN_CTL.bit.IE0= 1;
351 CanbRegs.CAN_CTL.bit.EIE= 1;
352 // ----- code for CAN end here -----
353
354 // Clear all interrupts and initialize PIE vector table:
355 // Disable CPU interrupts
356 DINT;
357
358 // Initialize the PIE control registers to their default state.
359 // The default state is all PIE interrupts disabled and flags
360 // are cleared.
361 // This function is found in the F2837xD_PieCtrl.c file.
362 InitPieCtrl();
363
364 // Disable CPU interrupts and clear all CPU interrupt flags:
365 IER = 0x0000;
366 IFR = 0x0000;
367
368 // Initialize the PIE vector table with pointers to the shell Interrupt
369 // Service Routines (ISR).
370 // This will populate the entire table, even if the interrupt
371 // is not used in this example. This is useful for debug purposes.
372 // The shell ISR routines are found in F2837xD_DefaultIsr.c.
373 // This function is found in F2837xD_PieVect.c.
374 InitPieVectTable();
375
376 // Interrupts that are used in this example are re-mapped to
377 // ISR functions found within this project
378 EALLOW; // This is needed to write to EALLOW protected registers
379 PieVectTable.TIMER0_INT = &cpu_timer0_isr;
380 PieVectTable.TIMER1_INT = &cpu_timer1_isr;
381 PieVectTable.TIMER2_INT = &cpu_timer2_isr;
382 PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
383 PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
384 PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
385 PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
386 PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
387 PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
388 PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
389 PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
390 PieVectTable.ADCCL1_INT = &ADCC_ISR;
391 PieVectTable.SPIB_RX_INT = &SPIB_isr;
392 PieVectTable.EMIF_ERROR_INT = &SWI1_HighestPriority; // Using Interrupt12 interrupts that are not used as SWIs
393 PieVectTable.RAM_CORRECTABLE_ERROR_INT = &SWI2_MiddlePriority;
394 PieVectTable.FLASH_CORRECTABLE_ERROR_INT = &SWI3_LowestPriority;
395 // ----- code for CAN start here -----
396 PieVectTable.CANB0_INT = &can_isr;
397 // ----- code for CAN end here -----
398 EDIS; // This is needed to disable write to EALLOW protected registers
399
400 // Initialize the CpuTimers Device Peripheral. This function can be
401 // found in F2837xD_CpuTimers.c
402 InitCpuTimers();
403
404 // Configure CPU-Timer 0, 1, and 2 to interrupt every second:
405 // 200MHz CPU Freq, Period (in uSeconds)
406 ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000); // Currently not used for any purpose
407 ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 100000); // !!!! Important, Used to command LADAR every 100ms. Do not Change.
408 ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 40000); // Currently not used for any purpose
409
410 // Enable CpuTimer Interrupt bit TIE
411 CpuTimer0Regs.TCR.all = 0x4000;
412 CpuTimer1Regs.TCR.all = 0x4000;
413 CpuTimer2Regs.TCR.all = 0x4000;
414
415 DELAY_US(1000000); // Delay 1 second giving Lidar Time to power on after system power on
416
417 init_serialSCIB(&SerialB,19200);
418 init_serialSCIC(&SerialC,19200);
419
420 for (LADARi = 0; LADARi < 228; LADARi++) {
421     ladar_data[LADARi].angle = ((3*LADARi+44)*0.3515625-135)*0.01745329; //0.017453292519943 is pi/180, multiplication is faster; 0.3515625 is 360/1024
422 }
423
424 init_eQEPs();
425 init_EPWM1and2();
426 init_RCServoPWM_3AB_5AB_6A();
427 init_ADCsAndDACs();
428 setupSpib();
429
430 EALLOW;
431 EPwm4Regs.ETSEL.bit.SOCAEN = 0; // Disable SOC on A group
432 EPwm4Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
433 EPwm4Regs.ETSEL.bit.SOCASEL = 2; // Select Event when counter equal to PRD
434 EPwm4Regs.ETPS.bit.SOCAPRD = 1; // Generate pulse on 1st event
435 EPwm4Regs.TBCTR = 0x0; // Clear counter
436 EPwm4Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
437 EPwm4Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
438 EPwm4Regs.TBCTL.bit.CLKDIV = 0; // divide by 1 50Mhz Clock
439 EPwm4Regs.TBPRD = 50000; // Set Period to 1ms sample. Input clock is 50MHz.
440 // Notice here that we are not setting CMPA or CMPB because we are not using the PWM signal
441 EPwm4Regs.ETSEL.bit.SOCAEN = 1; //enable SOCA
442 //EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, wait to do this right before enabling interrupts
443 EDIS;
444
445 // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
446 // which is connected to CPU-Timer 1, and CPU int 14, which is connected
447 // to CPU-Timer 2: int 12 is for the SWI.
448 IER |= M_INT1;
449 IER |= M_INT6;
450 IER |= M_INT8; // SCIC SCID
451 IER |= M_INT9; // SCIA
452 IER |= M_INT12;
453 IER |= M_INT13;
454 IER |= M_INT14;

```

```

455 // Enable TINT0 in the PIE: Group 1 interrupt 7
456 PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
457 PieCtrlRegs.PIEIER1.bit.INTx3 = 1;
458 PieCtrlRegs.PIEIER6.bit.INTx3 = 1;
459 PieCtrlRegs.PIEIER12.bit.INTx9 = 1; //SWI1
460 PieCtrlRegs.PIEIER12.bit.INTx10 = 1; //SWI2
461 PieCtrlRegs.PIEIER12.bit.INTx11 = 1; //SWI3 Lowest priority
462 // ----- code for CAN start here -----
463 // Enable CANB in the PIE: Group 9 interrupt 7
464 PieCtrlRegs.PIEIER9.bit.INTx7 = 1;
465 // ----- code for CAN end here -----
466
467 // ----- code for CAN start here -----
468 // Enable the CAN interrupt signal
469 CanbRegs.CAN_GLB_INT_EN.bit.GLBINT0_EN = 1;
470 // ----- code for CAN end here -----
471
472
473
474 robotdest[0].x = -4;    robotdest[0].y = 10;
475 robotdest[1].x = -4;    robotdest[1].y = 2;
476 //middle of bottom
477 robotdest[2].x = 0;     robotdest[2].y = 2;
478 //outside the course
479 robotdest[3].x = 0;     robotdest[3].y = -3;
480 //back to middle
481 robotdest[4].x = 0;     robotdest[4].y = 2;
482 robotdest[5].x = 4;     robotdest[5].y = 2;
483 robotdest[6].x = 4;     robotdest[6].y = 10;
484 robotdest[7].x = 0;     robotdest[7].y = 9;
485 //cjca added two points
486 robotdest[8].x = 4;     robotdest[8].y = 5;
487 robotdest[9].x = 2;     robotdest[9].y = 8;
488
489 // ROBOTps will be updated by Optitrack during gyro calibration
490 // TODO: specify the starting position of the robot
491 ROBOTps.x = 0;          //the estimate in array form (useful for matrix operations)
492 ROBOTps.y = 0;
493 ROBOTps.theta = 0;      // was -PI: need to flip OT ground plane to fix this
494 x_pred[0][0] = ROBOTps.x; //estimate in structure form (useful elsewhere)
495 x_pred[1][0] = ROBOTps.y;
496 x_pred[2][0] = ROBOTps.theta;
497
498 AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
499 PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
500 SpibRegs.SPIFFRX.bit.RXFFOVFCLR=1; // Clear Overflow flag
501 SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
502 PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
503 EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, and enter up count mode
504
505 init_serialSCIA(&SerialA,115200);
506 init_serialSCID(&SerialD,2083332);
507
508 // Enable global Interrupts and higher priority real-time debug events
509 EINT; // Enable Global interrupt INTM
510 ERTM; // Enable Global realtime interrupt DBGM
511 // ----- code for CAN start here -----
512 // Measured Distance from 1
513 // Initialize the receive message object 1 used for receiving CAN messages.
514 // Message Object Parameters:
515 //   Message Object ID Number: 1
516 //   Message Identifier: 0x060b0101
517 //   Message Frame: Standard
518 //   Message Type: Receive
519 //   Message ID Mask: 0x0
520 //   Message Object Flags: Receive Interrupt
521 //   Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
522 //   for a Receive mailbox)
523 //
524 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_1, 0x060b0101, CAN_MSG_FRAME_EXT,
525                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
526                       RX_MSG_DATA_LENGTH);
527
528 // Measured Distance from 2
529 // Initialize the receive message object 2 used for receiving CAN messages.
530 // Message Object Parameters:
531 //   Message Object ID Number: 2
532 //   Message Identifier: 0x060b0102
533 //   Message Frame: Standard
534 //   Message Type: Receive
535 //   Message ID Mask: 0x0
536 //   Message Object Flags: Receive Interrupt
537 //   Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
538 //   for a Receive mailbox)
539 //
540 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_2, 0x060b0102, CAN_MSG_FRAME_EXT,
541                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
542                       RX_MSG_DATA_LENGTH);
543
544 // Measurement Quality from 1
545 // Initialize the receive message object 2 used for receiving CAN messages.
546 // Message Object Parameters:
547 //   Message Object ID Number: 3
548 //   Message Identifier: 0x060b0201
549 //   Message Frame: Standard
550 //   Message Type: Receive
551 //   Message ID Mask: 0x0
552 //   Message Object Flags: Receive Interrupt
553 //   Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
554 //   for a Receive mailbox)
555 //
556 //
557 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_3, 0x060b0201, CAN_MSG_FRAME_EXT,
558                       CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
559                       RX_MSG_DATA_LENGTH);
560
561 // Measurement Quality from 2
562 // Initialize the receive message object 2 used for receiving CAN messages.
563 // Message Object Parameters:
564 //   Message Object ID Number: 4
565 //   Message Identifier: 0x060b0202
566 //   Message Frame: Standard
567 //   Message Type: Receive
568

```

```

569 //      Message ID Mask: 0x0
570 //      Message Object Flags: Receive Interrupt
571 //      Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
572 //      for a Receive mailbox)
573 //
574
575 CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_4, 0x060b0202, CAN_MSG_FRAME_EXT,
576                      CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
577                      RX_MSG_DATA_LENGTH);
578
579 //
580 // Start CAN module operations
581 //
582 CanbRegs.CAN_CTL.bit.Init = 0;
583 CanbRegs.CAN_CTL.bit.CCE = 0;
584
585
586 // ----- code for CAN end here -----
587 char S_command[19] = "S1152000124000\n"; // this change the baud rate to 115200
588 uint16_t S_len = 19;
589 serial_sendSCIC(&SerialC, S_command, S_len);
590
591
592 DELAY_US(1000000); // Delay letting Lidar change its Baud rate
593 init_serialSCIC(&SerialC, 115200);
594
595
596 // LED1 is GPIO22
597 GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
598 // LED2 is GPIO94
599 GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
600 // LED3 is GPIO95
601 GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
602 // LED4 is GPIO97
603 GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
604 // LED5 is GPIO111
605 GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
606
607
608 // IDLE loop. Just sit and loop forever (optional):
609 while(1)
610 {
611     if (UARTPrint == 1) {
612
613         if (readbuttons() == 0) {
614             UART_printfLine(1, "State:%d : %d", RobotState, statePos);
615             UART_printfLine(2, "0 %.3f G %.3f", odist, gdist);
616             //UART_printfLine(1, "Vrf:%.2f trn:%.2f", vref, turn);
617             //UART_printfLine(1, "x:%.2f y:%.2f a:%.2f", ROBOTps.x, ROBOTps.y, ROBOTps.theta);
618             UART_printfLine(2, "F%.4f R%.4f", LADARfront, LADARrightfront);
619         } else if (readbuttons() == 1) {
620             UART_printfLine(1, "01A:%.0fC:%.0fR:%.0f", MaxAreaThreshold1, MaxColThreshold1, MaxRowThreshold1);
621             UART_printfLine(2, "P1A:%.0fC:%.0fR:%.0f", MaxAreaThreshold2, MaxColThreshold2, MaxRowThreshold2);
622             //UART_printfLine(1, "LV1:%.3f LV2:%.3f", printLV1, printLV2);
623             //UART_printfLine(2, "Ln1:%.3f Ln2:%.3f", printLinux1, printLinux2);
624         } else if (readbuttons() == 2) {
625             UART_printfLine(1, "02A:%.0fC:%.0fR:%.0f", NextLargestAreaThreshold1, NextLargestColThreshold1, NextLargestRowThreshold1);
626             UART_printfLine(2, "P2A:%.0fC:%.0fR:%.0f", NextLargestAreaThreshold2, NextLargestColThreshold2, NextLargestRowThreshold2);
627             //UART_printfLine(1, "%.2f %.2f", adcC2Volt, adcC3Volt);
628             //UART_printfLine(2, "%.2f %.2f", adcC4Volt, adcC5Volt);
629         } else if (readbuttons() == 4) {
630             UART_printfLine(1, "03A:%.0fC:%.0fR:%.0f", NextNextLargestAreaThreshold1, NextNextLargestColThreshold1, NextNextLargestRowThreshold1);
631             UART_printfLine(2, "P3A:%.0fC:%.0fR:%.0f", NextNextLargestAreaThreshold2, NextNextLargestColThreshold2, NextNextLargestRowThreshold2);
632             //UART_printfLine(1, "L:%.3f R:%.3f", LeftVel, RightVel);
633             //UART_printfLine(2, "uL:%.2f uR:%.2f", uLeft, uRight);
634         } else if (readbuttons() == 8) {
635             UART_printfLine(1, "020x%.2f y%.2f", ladar_pts[20].x, ladar_pts[20].y);
636             UART_printfLine(2, "150x%.2f y%.2f", ladar_pts[150].x, ladar_pts[150].y);
637         } else if (readbuttons() == 3) {
638             UART_printfLine(1, "Vrf:%.2f trn:%.2f", vref, turn);
639             UART_printfLine(2, "MPU:%.2f LPR:%.2f", gyro9250_radians, gyroLPR510_radians);
640         } else if (readbuttons() == 5) {
641             UART_printfLine(1, "0x:%.2f 0y:%.2f 0a:%.2f", OPTITRACKps.x, OPTITRACKps.y, OPTITRACKps.theta);
642             UART_printfLine(2, "State:%d : %d", RobotState, statePos);
643         } else if (readbuttons() == 6) {
644             UART_printfLine(1, "D1 %ld D2 %ld", dis_1, dis_2);
645             UART_printfLine(2, "St1 %ld St2 %ld", measure_status_1, measure_status_2);
646         } else if (readbuttons() == 7) {
647             UART_printfLine(1, "%.0f, %.1f, %.1f, %.1f", tagid, tagx, tagy, tagz);
648             UART_printfLine(2, "%.1f, %.1f, %.1f", tagthetax, tagthetay, tagthetaz);
649         }
650         //UART_printfLine(1, "RCangle %.1f ", RCangle);
651
652         UARTPrint = 0;
653     }
654 }
655 }
656
657 __interrupt void SPIB_isr(void){
658
659     uint16_t i;
660     GpioDataRegs.GPBSET.bit.GPIO32 = 1;
661
662     GpioDataRegs.GPCSET.bit.GPIO66 = 1; //Pull CS high as done R/W
663     GpioDataRegs.GPASET.bit.GPIO29 = 1; // Pull CS high for DAN28027
664
665     if (CurrentChip == MPU9250) {
666
667         for (i=0; i<8; i++) {
668             readdata[i] = SpibRegs.SPIRXBUF; // readdata[0] is garbage
669         }
670
671         PostSWI1(); // Manually cause the interrupt for the SWI1
672
673     } else if (CurrentChip == DAN28027) {
674
675         DAN28027Garbage = SpibRegs.SPIRXBUF;
676         dan28027adc1 = SpibRegs.SPIRXBUF;
677         dan28027adc2 = SpibRegs.SPIRXBUF;
678         CurrentChip = MPU9250;
679         SpibRegs.SPIFFCT.bit.TXDLY = 0;
680     }
681
682     SpibRegs.SPIFFRX.bit.RXFFOVFLCLR=1; // Clear Overflow flag

```



```

683     SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
684     PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
685     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
686 }
687
688
689 //adcd1 pie interrupt
690 __interrupt void ADCC_ISR (void)
691 {
692     GpioDataRegs.GPASET.bit.GPIO11 = 1;
693     adcc2result = AdccResultRegs.ADCRESULT0;
694     adcc3result = AdccResultRegs.ADCRESULT1;
695     adcc4result = AdccResultRegs.ADCRESULT2;
696     adcc5result = AdccResultRegs.ADCRESULT3;
697
698     // Here covert ADCIND0 to volts
699     adcC2Volt = adcc2result*3.0/4095.0;
700     adcC3Volt = adcc3result*3.0/4095.0;
701     adcC4Volt = adcc4result*3.0/4095.0;
702     adcC5Volt = adcc5result*3.0/4095.0;
703
704     if (MPU9250ignoreCNT >= 1) {
705         CurrentChip = MPU9250;
706         SpibRegs.SPIFFCT.bit.TXDLY = 0;
707         SpibRegs.SPIFFRX.bit.RXFFIL = 8;
708
709         GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;
710
711         SpibRegs.SPITXBUF = ((0x8000)|(0x3A00));
712         SpibRegs.SPITXBUF = 0;
713         SpibRegs.SPITXBUF = 0;
714         SpibRegs.SPITXBUF = 0;
715         SpibRegs.SPITXBUF = 0;
716         SpibRegs.SPITXBUF = 0;
717         SpibRegs.SPITXBUF = 0;
718         SpibRegs.SPITXBUF = 0;
719     } else {
720         MPU9250ignoreCNT++;
721     }
722
723     numADCCcalls++;
724     AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1; //clear interrupt flag
725     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
726     GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;
727 }
728 }
729
730 // cpu_timer0_isr - CPU Timer0 ISR
731 __interrupt void cpu_timer0_isr(void)
732 {
733     CpuTimer0.InterruptCount++;
734
735     numTimer0calls++;
736
737     if ((numTimer0calls%5) == 0) {
738         // Blink LaunchPad Red LED
739         //GpioDataRegs.GPBTGGLE.bit.GPIO34 = 1;
740     }
741
742     // Acknowledge this interrupt to receive more interrupts from group 1
743     PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
744 }
745
746 // cpu_timer1_isr - CPU Timer1 ISR
747 __interrupt void cpu_timer1_isr(void)
748 {
749
750     serial_sendSCIC(&SerialC, G_command, G_len);
751
752     CpuTimer1.InterruptCount++;
753 }
754
755 // cpu_timer2_isr CPU Timer2 ISR
756 __interrupt void cpu_timer2_isr(void)
757 {
758     // Blink LaunchPad Blue LED
759     //GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
760
761     CpuTimer2.InterruptCount++;
762
763     PostSWI3();
764
765
766     // if ((CpuTimer2.InterruptCount % 10) == 0) {
767     //     UARTPrint = 1;
768     // }
769 }
770
771 void setF28027EPWM1A(float controleffort){
772     if (controleffort < -10) {
773         controleffort = -10;
774     }
775     if (controleffort > 10) {
776         controleffort = 10;
777     }
778     float value = (controleffort+10)*3000.0/20.0;
779     EPwm1A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
780 }
781 void setF28027EPWM2A(float controleffort){
782     if (controleffort < -10) {
783         controleffort = -10;
784     }
785     if (controleffort > 10) {
786         controleffort = 10;
787     }
788     float value = (controleffort+10)*3000.0/20.0;
789     EPwm2A_F28027 = (int16_t)value; // Set global variable that is sent over SPI to F28027
790 }
791
792 //
793 // Connected to PIEIER12_9 (use MINT12 and MG12_9 masks):
794 //
795 __interrupt void SWI1_HighestPriority(void) // EMIF_ERROR
796 {

```

```

797 GpioDataRegs.GPBSET.bit.GPIO61 = 1;
798 // Set interrupt priority:
799 volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
800 IER |= M_INT12;
801 IER &= MINT12; // Set "global" priority
802 PieCtrlRegs.PIEIER12.all &= MG12_9; // Set "group" priority
803 PieCtrlRegs.PIEACK.all = 0xFFFF; // Enable PIE interrupts
804 __asm(" NOP");
805 EINT;
806
807 //#####
808 IMU_data[0] = readdata[1];
809 IMU_data[1] = readdata[2];
810 IMU_data[2] = readdata[3];
811 IMU_data[3] = readdata[5];
812 IMU_data[4] = readdata[6];
813 IMU_data[5] = readdata[7];
814
815 accelx = (((float)(IMU_data[0]))*4.0/32767.0);
816 accely = (((float)(IMU_data[1]))*4.0/32767.0);
817 accelz = (((float)(IMU_data[2]))*4.0/32767.0);
818 gyrox = (((float)(IMU_data[3]))*250.0/32767.0);
819 gyroy = (((float)(IMU_data[4]))*250.0/32767.0);
820 gyroz = (((float)(IMU_data[5]))*250.0/32767.0);
821
822 if(calibration_state == 0){
823     calibration_count++;
824     if (calibration_count == 2000) {
825         calibration_state = 1;
826         calibration_count = 0;
827     }
828 } else if(calibration_state == 1){
829     accelx_offset+=accelx;
830     accely_offset+=accely;
831     accelz_offset+=accelz;
832     gyrox_offset+=gyrox;
833     gyroy_offset+=gyroy;
834     gyroz_offset+=gyroz;
835     gyroLPR510_offset+=adcC5Volt;
836     calibration_count++;
837     if (calibration_count == 2000) {
838         calibration_state = 2;
839         accelx_offset/=2000.0;
840         accely_offset/=2000.0;
841         accelz_offset/=2000.0;
842         gyrox_offset/=2000.0;
843         gyroy_offset/=2000.0;
844         gyroz_offset/=2000.0;
845         gyroLPR510_offset/=2000.0;
846         calibration_count = 0;
847     }
848 } else if(calibration_state == 2) {
849
850     gyroLPR510_drift += (((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005)/(2000);
851     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
852
853     gyro9250_drift += (((gyroz-gyroz_offset) + old_gyro9250)*.0005)/(2000);
854     old_gyro9250 = gyroz-gyroz_offset;
855     calibration_count++;
856
857     if (calibration_count == 2000) {
858         calibration_state = 3;
859         calibration_count = 0;
860         doneCal = 1;
861         newOPTITRACKpose = 0;
862     }
863 } else if(calibration_state == 3){
864     accelx -=(accelx_offset);
865     accely -=(accely_offset);
866     accelz -=(accelz_offset);
867     gyrox -= gyrox_offset;
868     gyroy -= gyroy_offset;
869     gyroz -= gyroz_offset;
870     LeftWheel = readEncLeft();
871     RightWheel = readEncRight();
872     HandValue = readEncWheel();
873
874     gyro9250_angle = gyro9250_angle + (gyroz + old_gyro9250)*.0005 - gyro9250_drift;
875     old_gyro9250 = gyroz;
876
877     gyroLPR510_angle = gyroLPR510_angle + ((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005 - gyroLPR510_drift;
878     old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;
879
880     gyro9250_radians = (gyro9250_angle * (PI/180.0));
881     gyroLPR510_radians = gyroLPR510_angle * 400 * (PI/180.0);
882
883     LeftVel = (1.235/12.0)*(LeftWheel - LeftWheel_1)*1000;
884     RightVel = (1.235/12.0)*(RightWheel - RightWheel_1)*1000;
885
886     if (newLinuxCommands == 1) {
887         newLinuxCommands = 0;
888         printLinux1 = LinuxCommands[0];
889         printLinux2 = LinuxCommands[1];
890         /// = LinuxCommands[2];
891         /// = LinuxCommands[3];
892         /// = LinuxCommands[4];
893         /// = LinuxCommands[5];
894         /// = LinuxCommands[6];
895         /// = LinuxCommands[7];
896         /// = LinuxCommands[8];
897         /// = LinuxCommands[9];
898         /// = LinuxCommands[10];
899     }
900
901     if (NewCAMDataThreshold1 == 1) {
902         NewCAMDataThreshold1 = 0;
903         MaxAreaThreshold1 = fromCAMvaluesThreshold1[0];
904         MaxColThreshold1 = fromCAMvaluesThreshold1[1];
905         MaxRowThreshold1 = fromCAMvaluesThreshold1[2];
906
907         NextLargestAreaThreshold1 = fromCAMvaluesThreshold1[3];
908         NextLargestColThreshold1 = fromCAMvaluesThreshold1[4];
909         NextLargestRowThreshold1 = fromCAMvaluesThreshold1[5];
910

```



```

911     NextNextLargestAreaThreshold1 = fromCAMvaluesThreshold1[6];
912     NextNextLargestColThreshold1 = fromCAMvaluesThreshold1[7];
913     NextNextLargestRowThreshold1 = fromCAMvaluesThreshold1[8];
914     numThres1++;
915
916     // cjca Fit a third order polynomial in matlab to convert from (orange) row threshold values to depth from the camera (in)
917     float mr = MaxRowThreshold1;
918     odist = -1.15291070692441e-05*pow(mr,3)+0.00348114123702629*pow(mr,2)-0.365334496447587*pow(mr,1)+13.7577551982332+0.75;
919     if ((numThres1 % 5) == 0) {
920         // LED4 is GPIO97
921         GpioDataRegs.GPDT0GGLE.bit.GPIO97 = 1;
922     }
923     if ((numThres1 % 100) == 0) {
924         UARTPrint = 1;
925     }
926 }
927
928 if (NewCAMDataThreshold2 == 1) {
929     NewCAMDataThreshold2 = 0;
930     MaxAreaThreshold2 = fromCAMvaluesThreshold2[0];
931     MaxColThreshold2 = fromCAMvaluesThreshold2[1];
932     MaxRowThreshold2 = fromCAMvaluesThreshold2[2];
933
934     NextLargestAreaThreshold2 = fromCAMvaluesThreshold2[3];
935     NextLargestColThreshold2 = fromCAMvaluesThreshold2[4];
936     NextLargestRowThreshold2 = fromCAMvaluesThreshold2[5];
937
938     NextNextLargestAreaThreshold2 = fromCAMvaluesThreshold2[6];
939     NextNextLargestColThreshold2 = fromCAMvaluesThreshold2[7];
940     NextNextLargestRowThreshold2 = fromCAMvaluesThreshold2[8];
941     numThres2++;
942
943     // cjca Apply the same polynomial fit to green objects
944     float mr = MaxRowThreshold2;
945     gdist = -1.15291070692441e-05*pow(mr,3)+0.00348114123702629*pow(mr,2)-0.365334496447587*pow(mr,1)+13.7577551982332+0.75;
946
947     if ((numThres2 % 5) == 0) {
948         // LED5 is GPIO111
949         GpioDataRegs.GPDT0GGLE.bit.GPIO111 = 1;
950     }
951     if ((numThres2 % 100) == 0) {
952         UARTPrint = 1;
953     }
954 }
955
956 if (NewCAMDataAprilTag1 == 1) {
957     NewCAMDataAprilTag1 = 0;
958     tagx = fromCAMvaluesAprilTag1[0];
959     tagy = fromCAMvaluesAprilTag1[1];
960     tagz = fromCAMvaluesAprilTag1[2];
961
962     tagthetax = fromCAMvaluesAprilTag1[3];
963     tagthetay = fromCAMvaluesAprilTag1[4];
964     tagthetaz = fromCAMvaluesAprilTag1[5];
965
966     tagid = fromCAMvaluesAprilTag1[6];
967
968     numtag++;
969     if ((numtag % 5) == 0) {
970         // LED2 is GPIO94
971         GpioDataRegs.GPCT0GGLE.bit.GPIO94 = 1;
972     }
973 }
974
975 if (NewLVData == 1) {
976     NewLVData = 0;
977     printLV1 = fromLVvalues[0];
978     printLV2 = fromLVvalues[1];
979     ///? = fromLVvalues[2];
980     ///? = fromLVvalues[3];
981     ///? = fromLVvalues[4];
982     ///? = fromLVvalues[5];
983     ///? = fromLVvalues[6];
984     ///? = fromLVvalues[7];
985 }
986
987
988 if (new_optitrack == 1) {
989     OPTITRACKps = UpdateOptitrackStates(ROBOTps, &newOPTITRACKpose);
990     new_optitrack = 0;
991 }
992
993 // Step 0: update B, u
994 B[0][0] = cosf(ROBOTps.theta)*0.001;
995 B[1][0] = sinf(ROBOTps.theta)*0.001;
996 B[2][1] = 0.001;
997 u[0][0] = 0.5*(LeftVel + RightVel); // linear velocity of robot
998 u[1][0] = gyroz*(PI/180.0); // angular velocity in rad/s (negative for right hand angle)
999
1000 // Step 1: predict the state and estimate covariance
1001 Matrix3x2 Mult(B, u, Bu); // Bu = B*u
1002 Matrix3x1 Add(x_pred, Bu, x_pred, 1.0, 1.0); // x_pred = x_pred(old) + Bu
1003 Matrix3x3 Add(P_pred, Q, P_pred, 1.0, 1.0); // P_pred = P_pred(old) + Q
1004 // Step 2: if there is a new measurement, then update the state
1005 if (1 == newOPTITRACKpose) {
1006     OptiNumUpdatesProcessed++; // checking how often OptiTrack is causing a Kalman update
1007     if ((OptiNumUpdatesProcessed%10)==0) {
1008         // LED 3
1009         GpioDataRegs.GPCT0GGLE.bit.GPIO95 = 1;
1010     }
1011     newOPTITRACKpose = 0;
1012     z[0][0] = OPTITRACKps.x; // take in the Optitrack Motion Capture measurement
1013     z[1][0] = OPTITRACKps.y;
1014     // fix for OptiTrack problem at 180 degrees
1015     if (cosf(ROBOTps.theta) < -0.99) {
1016         z[2][0] = ROBOTps.theta;
1017     }
1018     else {
1019         z[2][0] = OPTITRACKps.theta;
1020     }
1021     // Step 2a: calculate the innovation/measurement residual, ytilde
1022     Matrix3x1 Add(z, x_pred, ytilde, 1.0, -1.0); // ytilde = z-x_pred
1023     // Step 2b: calculate innovation covariance, S
1024     Matrix3x3 Add(P_pred, R, S, 1.0, 1.0); // S = P_pred + R

```

```

1025 // Step 2c: calculate the optimal Kalman gain, K
1026 Matrix3x3_Invert(S, S_inv);
1027 Matrix3x3_Mult(P_pred, S_inv, K); // K = P_pred*(S^-1)
1028 // Step 2d: update the state estimate x_pred = x_pred(old) + K*ytilde
1029 Matrix3x1_Mult(K, ytilde, temp_3x1);
1030 Matrix3x1_Add(x_pred, temp_3x1, x_pred, 1.0, 1.0);
1031 // Step 2e: update the covariance estimate P_pred = (I-K)*P_pred(old)
1032 Matrix3x3_Add(eye3, K, temp_3x3, 1.0, -1.0);
1033 Matrix3x3_Mult(temp_3x3, P_pred, P_pred);
1034 } // end of correction step
1035
1036 // set ROBOTps to the updated and corrected Kalman values.
1037 ROBOTps.x = x_pred[0][0];
1038 ROBOTps.y = x_pred[1][0];
1039 ROBOTps.theta = x_pred[2][0];
1040
1041 // Make sure this function is called every time in this function even if you decide not to use its vref and turn
1042 // uses xy code to step through an array of positions
1043 if( xy_control(&vref, &turn, 1.0, ROBOTps.x, ROBOTps.y, robotdest[statePos].x, robotdest[statePos].y, ROBOTps.theta, 0.25, 0.5)) {
1044     statePos = (statePos+1)%NUMWAYPOINTS;
1045 }
1046 // state machine
1047 //RobotState = 20;
1048 switch (RobotState) {
1049 case 1:
1050
1051     // vref and turn are the vref and turn returned from xy_control
1052
1053     if (LADARfront < 1.2) {
1054         vref = 0.2;
1055         checkfronttally++;
1056         if (checkfronttally > 310) { // check if LADARfront < 1.2 for 310ms or 3 LADAR samples
1057             RobotState = 10; // Wall follow
1058             WallFollowtime = 0;
1059             right_wall_follow_state = 1;
1060         }
1061     } else {
1062         checkfronttally = 0;
1063     }
1064     //cjca cause a 2 second delay when resetting
1065     count1 += 1;
1066     if (count1 > 2000) {
1067         //cjca decide which colored ball is closer
1068         if (MaxAreaThreshold1 > 35 & (MaxAreaThreshold1 > MaxAreaThreshold2)) {
1069             RobotState=20;
1070             //cjca also checks threshold of other ball as well
1071         } else if (MaxAreaThreshold2 > 35 & (MaxAreaThreshold2 > MaxAreaThreshold1)) {
1072             RobotState=30;
1073         }
1074     }
1075     break;
1076 case 10:
1077     if (right_wall_follow_state == 1) {
1078         //Left Turn
1079         turn = Kp_front_wall*(14.5 - LADARfront);
1080         vref = front_turn_velocity;
1081         if (LADARfront > left_turn_Stop_threshold) {
1082             right_wall_follow_state = 2;
1083         }
1084     } else if (right_wall_follow_state == 2) {
1085         //Right Wall Follow
1086         turn = Kp_right_wal*(ref_right_wall - LADARrightfront);
1087         vref = foward_velocity;
1088         if (LADARfront < left_turn_Start_threshold) {
1089             right_wall_follow_state = 1;
1090         }
1091     }
1092     if (turn > turn_saturation) {
1093         turn = turn_saturation;
1094     }
1095     if (turn < -turn_saturation) {
1096         turn = -turn_saturation;
1097     }
1098
1099     WallFollowtime++;
1100     if ( (WallFollowtime > 5000) && (LADARfront > 1.5) ) {
1101         RobotState = 1;
1102         checkfronttally = 0;
1103     }
1104     break;
1105 case 20:
1106     // put vision code here
1107     //cjca orange ball detected
1108     kpvision = -0.05;
1109     colcentroid = MaxColThreshold1 - 80; // cjca Change threshold range from 0 to 160 to -80 to 80
1110     // cjca Wait for object to reappear in camera if lost
1111     if (MaxColThreshold1 == 0 || MaxAreaThreshold1 < 3) {
1112         vref = 0;
1113         turn = 0;
1114     } else {
1115         // cjca follow ball at constant speed and turn defined by P controller to keep ball centered in camera
1116         vref = 0.75;
1117         turn = kpvision*(0-colcentroid);
1118     }
1119
1120     if (MaxRowThreshold1 > 115) {
1121         RobotState = 22; // cjca Always transition to 22 from 20
1122         count22 = 0;
1123     }
1124     break;
1125 case 22:
1126     //cjca stop to let gripper door open and wait one second
1127     vref = 0;
1128     turn = 0;
1129     count22 += 1;
1130     if (count22 > 1000) {
1131         RobotState = 24; // cjca Always transition to 24 from 22
1132         count22 = 0;
1133     }
1134     break;
1135 case 24:
1136     //cjca Approach the ball slowly for one second to pick it up
1137     vref = 0.5;
1138

```

```

1139         turn = 0;
1140         count22 += 1;
1141         if (count22 > 1000) {
1142             RobotState = 26; // cjna Always transition to 26 from 24
1143             count22 = 0;
1144         }
1145         break;
1146     case 26:
1147         // cjna Stop with the collected for one second ball before going back to waypoint following
1148         vref = 0;
1149         turn = 0;
1150         count22 += 1;
1151         if (count22 > 1000) {
1152             RobotState = 1;
1153             count22 = 0;
1154             count1 = 0;
1155         }
1156         //cjna continue to x,y point
1157         break;
1158
1159
1160     case 30:
1161         // put vision code here
1162         //cjna do the same but for the green ball now
1163         kpvision = -0.05;
1164         colcentroid = MaxColThreshold2 - 80;
1165         if (MaxColThreshold2 == 0 || MaxAreaThreshold2 < 3) {
1166             vref = 0;
1167             turn = 0;
1168         } else {
1169             vref = 0.75;
1170             turn = kpvision*(0-colcentroid);
1171         }
1172
1173         if (MaxRowThreshold2 > 100) {
1174             RobotState = 32;
1175             count22 = 0;
1176         }
1177         break;
1178     ////cjna delay by a second each time to run smoother
1179     case 32:
1180         vref = 0;
1181         turn = 0;
1182         count22 += 1;
1183         if (count22 > 1000) {
1184             RobotState = 34;
1185             count22 = 0;
1186         }
1187         break;
1188     //cjna uses the same count variable for convenience
1189     case 34:
1190         vref = 0.5;
1191         turn = 0;
1192         count22 += 1;
1193         if (count22 > 1000) {
1194             RobotState = 36;
1195             count22 = 0;
1196         }
1197         break;
1198     case 36:
1199         vref = 0;
1200         turn = 0;
1201         count22 += 1;
1202         if (count22 > 1000) {
1203             RobotState = 1;
1204             count22 = 0;
1205             //cjna reset the count variable for 2 second delay
1206             count1 = 0;
1207         }
1208         break;
1209     default:
1210         break;
1211 }
1212
1213
1214 //Must be called each time into this SWI1 function
1215 PControl(&uLeft,&uRight,vref,turn,LeftWheel,RightWheel);
1216 // These below lines also must be called each time into this SWI1 function
1217 setPWM1A(uLeft);
1218 setPWM2A(-uRight);
1219 setF28027PWM1A(uLeft);
1220 setF28027PWM2A(-uRight);
1221 LeftWheel_1 = LeftWheel;
1222 RightWheel_1 = RightWheel;
1223
1224 if((timecount%250) == 0) {
1225     DataToLabView.floatData[0] = ROBOTps.x;
1226     DataToLabView.floatData[1] = ROBOTps.y;
1227     DataToLabView.floatData[2] = ROBOTps.theta;
1228     DataToLabView.floatData[3] = (float)timecount;
1229     DataToLabView.floatData[4] = 0.5*(LeftVel + RightVel);
1230     DataToLabView.floatData[5] = (float)RobotState;
1231     DataToLabView.floatData[6] = (float)statePos;
1232     DataToLabView.floatData[7] = LADARfront;
1233     LVsenddata[0] = '*'; // header for LVdata
1234     LVsenddata[1] = '$';
1235     for (i=0;i<LVNUM_TOFROM_FLOATS*4;i++) {
1236         if (i%2==0) {
1237             LVsenddata[i+2] = DataToLabView.rawData[i/2] & 0xFF;
1238         } else {
1239             LVsenddata[i+2] = (DataToLabView.rawData[i/2]>>8) & 0xFF;
1240         }
1241     }
1242     serial_sendSCID(&SerialID, LVsenddata, 4*LVNUM_TOFROM_FLOATS + 2);
1243 }
1244
1245 }
1246 timecount++;
1247 if((timecount%200) == 0)
1248 {
1249     if(doneCal == 0) {
1250         GpioDataRegs.GPATOGGLE.bit.GPI031 = 1; // Blink Blue LED while calibrating
1251     }
1252     GpioDataRegs.GPBTOGGLE.bit.GPI034 = 1; // Always Block Red LED

```

```

1253     UARTPrint = 1; // Tell While loop to print
1254 }
1255
1256 SpibRegs.SPIFFCT.bit.TXDLY = 16;
1257 CurrentChip = DAN28027;
1258 SpibRegs.SPIFFRX.bit.RXFFIL = 3;
1259 GpioDataRegs.GPACLEAR.bit.GPIO29 = 1;
1260 SpibRegs.SPITXBUF = 0x00DA;
1261 SpibRegs.SPITXBUF = EPwm1A_F28027;
1262 SpibRegs.SPITXBUF = EPwm2A_F28027;
1263
1264
1265 //#####
1266 //
1267 // Restore registers saved:
1268 //
1269 DINT;
1270 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1271
1272 GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
1273 }
1274
1275 //
1276 // Connected to PIEIER12_10 (use MINT12 and MG12_10 masks):
1277 //
1278 __interrupt void SWI2_MiddlePriority(void)    // RAM_CORRECTABLE_ERROR
1279 {
1280     // Set interrupt priority:
1281     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1282     IER |= M_INT12;
1283     IER &= MINT12;                // Set "global" priority
1284     PieCtrlRegs.PIEIER12.all &= MG12_10; // Set "group" priority
1285     PieCtrlRegs.PIEACK.all = 0xFFFF;    // Enable PIE interrupts
1286     __asm(" NOP");
1287     EINT;
1288
1289 //#####
1290 // Insert SWI ISR Code here.....
1291 // LED1
1292 GpioDataRegs.GPATOGGLE.bit.GPIO22 = 1;
1293 if (LADARpingpong == 1) {
1294     // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1295     LADARrightfront = 19; // 19 is greater than max feet
1296     for (LADARi = 52; LADARi <= 56 ; LADARi++) {
1297         if (ladar_data[LADARi].distance_ping < LADARrightfront) {
1298             LADARrightfront = ladar_data[LADARi].distance_ping;
1299         }
1300     }
1301     // LADARfront is the min of dist 111, 112, 113, 114, 115
1302     LADARfront = 19;
1303     for (LADARi = 111; LADARi <= 115 ; LADARi++) {
1304         if (ladar_data[LADARi].distance_ping < LADARfront) {
1305             LADARfront = ladar_data[LADARi].distance_ping;
1306         }
1307     }
1308     LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta));
1309     LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)+LADARps.y*cosf(ROBOTps.theta));
1310     for (LADARi = 0; LADARi < 228; LADARi++) {
1311
1312         ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_ping*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1313         ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_ping*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1314     }
1315 }
1316 } else if (LADARpingpong == 0) {
1317     // LADARrightfront is the min of dist 52, 53, 54, 55, 56
1318     LADARrightfront = 19; // 19 is greater than max feet
1319     for (LADARi = 52; LADARi <= 56 ; LADARi++) {
1320         if (ladar_data[LADARi].distance_pong < LADARrightfront) {
1321             LADARrightfront = ladar_data[LADARi].distance_pong;
1322         }
1323     }
1324     // LADARfront is the min of dist 111, 112, 113, 114, 115
1325     LADARfront = 19;
1326     for (LADARi = 111; LADARi <= 115 ; LADARi++) {
1327         if (ladar_data[LADARi].distance_pong < LADARfront) {
1328             LADARfront = ladar_data[LADARi].distance_pong;
1329         }
1330     }
1331     LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
1332     LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
1333     for (LADARi = 0; LADARi < 228; LADARi++) {
1334
1335         ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_pong*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
1336         ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_pong*sinf(ladar_data[LADARi].angle + ROBOTps.theta);
1337     }
1338 }
1339 }
1340
1341
1342
1343
1344 //#####
1345 //
1346 // Restore registers saved:
1347 //
1348 DINT;
1349 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1350
1351 }
1352
1353 //
1354 // Connected to PIEIER12_11 (use MINT12 and MG12_11 masks):
1355 //
1356 __interrupt void SWI3_LowestPriority(void)    // FLASH_CORRECTABLE_ERROR
1357 {
1358     // Set interrupt priority:
1359     volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
1360     IER |= M_INT12;
1361     IER &= MINT12;                // Set "global" priority
1362     PieCtrlRegs.PIEIER12.all &= MG12_11; // Set "group" priority
1363     PieCtrlRegs.PIEACK.all = 0xFFFF;    // Enable PIE interrupts
1364     __asm(" NOP");
1365     EINT;
1366

```

```

1367 //#####
1368 // Insert SWI ISR Code here.....
1369 //cjca enable the ewpm's for servo's
1370 RCangle = readEncWheel();
1371 setEPWM3A_RCServo(RCangle); //RCangle is a value from -90 to 90
1372 setEPWM3B_RCServo(RCangle); //RCangle is a value from -90 to 90
1373 setEPWM5A_RCServo(RCangle); //RCangle is a value from -90 to 90
1374 setEPWM5B_RCServo(RCangle); //RCangle is a value from -90 to 90
1375 setEPWM6A_RCServo(RCangle); //RCangle is a value from -90 to 90
1376
1377 //#####
1378 //
1379 // Restore registers saved:
1380 //
1381 DINT;
1382 PieCtrlRegs.PIEIER12.all = TempPIEIER;
1383
1384 }
1385
1386 // ----- code for CAN start here -----
1387 _interrupt void can_isr(void)
1388 {
1389     int i = 0;
1390
1391     uint32_t status;
1392
1393     //
1394     // Read the CAN interrupt status to find the cause of the interrupt
1395     //
1396     status = CANGetInterruptCause(CANB_BASE);
1397
1398     //
1399     // If the cause is a controller status interrupt, then get the status
1400     //
1401     if(status == CAN_INT_INT0ID_STATUS)
1402     {
1403         //
1404         // Read the controller status. This will return a field of status
1405         // error bits that can indicate various errors. Error processing
1406         // is not done in this example for simplicity. Refer to the
1407         // API documentation for details about the error status bits.
1408         // The act of reading this status will clear the interrupt.
1409         //
1410         status = CANgetStatus(CANB_BASE);
1411
1412     }
1413
1414     //
1415     // Check if the cause is the receive message object 2
1416     //
1417     else if(status == RX_MSG_OBJ_ID_1)
1418     {
1419         //
1420         // Get the received message
1421         //
1422         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_1, rxMsgData);
1423
1424         for(i = 0; i<2; i++)
1425         {
1426             dis_raw_1[i] = rxMsgData[i];
1427         }
1428
1429         dis_1 = 256*dis_raw_1[1] + dis_raw_1[0];
1430
1431         measure_status_1 = rxMsgData[2];
1432
1433         //
1434         // Getting to this point means that the RX interrupt occurred on
1435         // message object 2, and the message RX is complete. Clear the
1436         // message object interrupt.
1437         //
1438         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_1);
1439
1440         //
1441         // Increment a counter to keep track of how many messages have been
1442         // received. In a real application this could be used to set flags to
1443         // indicate when a message is received.
1444         //
1445         rxMsgCount_1++;
1446
1447         //
1448         // Since the message was received, clear any error flags.
1449         //
1450         errorFlag = 0;
1451     }
1452
1453     else if(status == RX_MSG_OBJ_ID_2)
1454     {
1455         //
1456         // Get the received message
1457         //
1458         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_2, rxMsgData);
1459
1460         for(i = 0; i<2; i++)
1461         {
1462             dis_raw_2[i] = rxMsgData[i];
1463         }
1464
1465         dis_2 = 256*dis_raw_2[1] + dis_raw_2[0];
1466
1467         measure_status_2 = rxMsgData[2];
1468
1469         //
1470         // Getting to this point means that the RX interrupt occurred on
1471         // message object 2, and the message RX is complete. Clear the
1472         // message object interrupt.
1473         //
1474         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_2);
1475
1476         //
1477         // Increment a counter to keep track of how many messages have been
1478         // received. In a real application this could be used to set flags to
1479         // indicate when a message is received.
1480         //

```

```

1481         rxMsgCount_2++;
1482
1483         //
1484         // Since the message was received, clear any error flags.
1485         //
1486         errorFlag = 0;
1487     }
1488
1489     else if(status == RX_MSG_OBJ_ID_3)
1490     {
1491         //
1492         // Get the received message
1493         //
1494         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_3, rxMsgData);
1495
1496         for(i = 0; i<4; i++)
1497         {
1498             lightlevel_raw_1[i] = rxMsgData[i];
1499             quality_raw_1[i] = rxMsgData[i+4];
1500         }
1501
1502         lightlevel_1 = ((256.0*256.0*256.0)*lightlevel_raw_1[3] + (256.0*256.0)*lightlevel_raw_1[2] + 256.0*lightlevel_raw_1[1] + lightlevel_raw_1[0])/65535;
1503         quality_1 = ((256.0*256.0*256.0)*quality_raw_1[3] + (256.0*256.0)*quality_raw_1[2] + 256.0*quality_raw_1[1] + quality_raw_1[0])/65535;
1504
1505
1506         //
1507         // Getting to this point means that the RX interrupt occurred on
1508         // message object 2, and the message RX is complete. Clear the
1509         // message object interrupt.
1510         //
1511         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_3);
1512
1513         //
1514         // Since the message was received, clear any error flags.
1515         //
1516         errorFlag = 0;
1517     }
1518
1519
1520
1521     else if(status == RX_MSG_OBJ_ID_4)
1522     {
1523         //
1524         // Get the received message
1525         //
1526         CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_4, rxMsgData);
1527
1528         for(i = 0; i<4; i++)
1529         {
1530             lightlevel_raw_2[i] = rxMsgData[i];
1531             quality_raw_2[i] = rxMsgData[i+4];
1532         }
1533
1534         lightlevel_2 = ((256.0*256.0*256.0)*lightlevel_raw_2[3] + (256.0*256.0)*lightlevel_raw_2[2] + 256.0*lightlevel_raw_2[1] + lightlevel_raw_2[0])/65535;
1535         quality_2 = ((256.0*256.0*256.0)*quality_raw_2[3] + (256.0*256.0)*quality_raw_2[2] + 256.0*quality_raw_2[1] + quality_raw_2[0])/65535;
1536
1537         //
1538         // Getting to this point means that the RX interrupt occurred on
1539         // message object 2, and the message RX is complete. Clear the
1540         // message object interrupt.
1541         //
1542         CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_4);
1543
1544         //
1545         // Since the message was received, clear any error flags.
1546         //
1547         errorFlag = 0;
1548     }
1549
1550
1551
1552
1553     //
1554     // If something unexpected caused the interrupt, this would handle it.
1555     //
1556     else
1557     {
1558         //
1559         // Spurious interrupt handling can go here.
1560         //
1561     }
1562
1563     //
1564     // Clear the global interrupt flag for the CAN interrupt line
1565     //
1566     CANclearGlobalInterruptStatus(CANB_BASE, CAN_GLOBAL_INT_CANINT0);
1567
1568     //
1569     // Acknowledge this interrupt located in group 9
1570     //
1571     InterruptclearACKGroup(INTERRUPT_ACK_GROUP9);
1572 }
1573 // ----- code for CAN end here -----

```